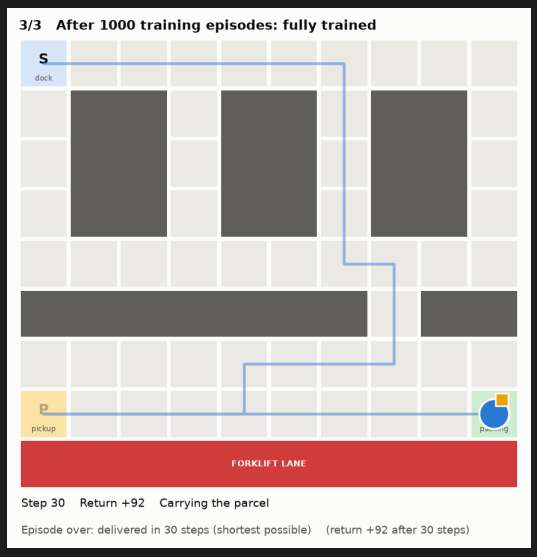
FUNDAMENTALS OF REINFORCEMENT LEARNING · ASSESSMENT TASK 2

Warehouse Delivery Robot

Learning Safe Parcel Delivery with Q-Learning and SARSA

Project report · RL Innovation Hackathon

A robot that learns, from rewards alone, to fetch a parcel and deliver it by the shortest safe route through a warehouse it has never been shown.



The trained robot: 30 moves, the shortest route

SUBMITTED BY · SECTION A

Karthik K S	RA2311026050027
Arun Prasath	RA2311026050057
Inzamam	RA2311026050098

COURSE	Fundamentals of Reinforcement Learning
ASSESSMENT	Task 2: RL Innovation Hackathon, build an intelligent learning agent
PROBLEM DOMAIN	Grid World
ALGORITHMS	Tabular Q-learning and SARSA, with a random-agent baseline
CODE	`github.com/KS-KARTHIK-05/Warehouse-Delivery-Robot-using-Q-Learning-and-SARSA`
DATE	October 2026

Abstract

This report presents an intelligent agent that learns, through reinforcement learning, to operate a delivery robot in a simulated warehouse. The robot must leave its charging dock, collect a parcel and deliver it to a packing station, while navigating around storage racks, passing through the single gate in a safety barrier and staying out of a forklift lane, where entry counts as a collision. We formulate the task as a Markov decision process with 106 states and four actions. The agent is never given the map: it learns from rewards alone using tabular Q-learning, and we compare it with SARSA and with a random baseline over ten independently seeded training runs per algorithm.

After about 300 training episodes, Q-learning delivers the parcel in every test using 30 moves, the shortest possible route as confirmed by a breadth-first search that the agent itself never uses. SARSA also delivers every time, but learns a 32-move route that keeps one row away from the forklift lane, reproducing the classic cliff-walking contrast between off-policy and on-policy learning: while still exploring, Q-learning collides in 15.6% of episodes and SARSA in 2.8%. A random agent never delivers. On a slippery floor where 10% of moves slide sideways, both algorithms learn the safer route and deliver in about 99% of tests. The complete system, with reproducible experiments, nine evaluation figures and four demonstration modes, uses only Python, NumPy and Matplotlib.

KEYWORDS reinforcement learning, Markov decision process, temporal-difference learning, Q-learning, SARSA, exploration, grid world, warehouse robotics

Contents

Abstract . . . 2
1 Introduction . . . 4
1.1 Motivation . . . 4
1.2 Objectives . . . 4
1.3 What we built . . . 4
1.4 Structure of this report . . . 5
2 Background . . . 5
2.1 Reinforcement learning . . . 5
2.2 Markov decision processes . . . 5
2.3 Action values and the Bellman optimality equation . . . 6
2.4 Temporal-difference learning . . . 6
2.5 Q-learning . . . 6
2.6 SARSA . . . 6
2.7 Exploration with ε-greedy . . . 7
2.8 On-policy vs. off-policy: the cliff-walking example . . . 7
3 Problem formulation . . . 7
3.1 Scenario . . . 7
3.2 Why this layout . . . 8
3.3 States, actions and transitions . . . 9
3.4 Reward design . . . 9
3.5 The Markov property and the carrying flag . . . 10
3.6 Termination and truncation . . . 10
3.7 Slippery-floor variant . . . 10
4 System design and implementation . . . 10
4.1 Architecture . . . 10
4.2 The environment . . . 11
4.3 The agents . . . 11
4.4 The interaction loop . . . 11
4.5 Experimental protocol . . . 12
4.6 Hyperparameters . . . 13
4.7 Visualisation and demonstration . . . 13
5 Results . . . 13
5.1 Final test performance . . . 14
5.2 Learning curves . . . 15
5.3 Steps per episode and success rate . . . 15
5.4 How training episodes ended . . . 16
5.5 Learning progress of a single robot . . . 17
5.6 Learned routes . . . 18
5.7 Inside the Q-tables . . . 19
5.8 Slippery floor . . . 20
6 Discussion . . . 21
6.1 Q-learning vs. SARSA: speed against safety . . . 21
6.2 Why reinforcement learning rather than path planning . . . 21
6.3 Evidence that the behaviour is learned . . . 21
6.4 Limitations . . . 21
7 Demonstration and usage guide . . . 22
7.1 Installation . . . 22
7.2 Demo modes . . . 22
7.3 What the demos look like . . . 23
8 Implemented features and future enhancements . . . 24
9 Conclusion . . . 24
References . . . 25
Appendix A Viva preparation: likely questions . . . 25
Appendix B Reproducibility details . . . 26
Appendix C Key source code . . . 27

1 Introduction

1.1 Motivation

Modern fulfilment centres move millions of parcels a day, and a growing share of that movement is done by autonomous mobile robots. A warehouse robot has two goals that pull against each other: it should be *fast*, because every extra metre costs time and battery, and it should be *safe*, because it shares the floor with people and forklifts. Programming every route by hand is brittle: when racks are moved or a new hazard zone appears, the routes have to be rewritten.

Reinforcement learning (RL) offers a different approach. Instead of being told what to do, the robot tries actions, receives rewards and penalties, and gradually learns which behaviour leads to the most reward over time. This project builds such an agent for a simplified but realistic warehouse, and studies how it learns, how fast, and how two classic RL algorithms differ in the trade-off between speed and safety.

1.2 Objectives

The project follows the brief of the RL Innovation Hackathon (Assessment Task 2). Its objectives were to:

- select a real-world decision-making problem and define its agent, environment, states, actions and rewards;
- select an appropriate RL algorithm and justify the choice;
- design and implement a working agent that learns through interaction rather than following a predefined solution;
- evaluate it with cumulative reward, success rate, number of steps and learning progress, using graphs and learning curves;
- demonstrate the working solution, and clearly separate implemented features from proposed future enhancements.

1.3 What we built

- **A warehouse environment** (a 9 × 10 grid world) with storage racks, a safety barrier with a single gate, a forklift-lane hazard, a two-stage pickup-and-delivery task and an optional slippery floor.
- **Two learning agents**, tabular Q-learning and SARSA, sharing every setting except their update rule, plus a random agent as a baseline.
- **A reproducible experiment**: ten seeded training runs per algorithm, regular tests with exploration switched off, and a final 100-episode test of every trained agent.
- **Nine figures and four live demonstrations** that show the learning as it happens, from a robot that wanders at random to one that drives the shortest route.

1.4 Structure of this report

Section 2 introduces the RL concepts the project relies on. Section 3 formulates the warehouse task as a Markov decision process. Section 4 describes the system design and the experimental protocol, and Section 5 presents the results. Section 6 discusses what they mean, including the limitations. Section 7 explains how to run the code and the demonstrations, Section 8 separates implemented features from future work, and Section 9 concludes. The appendices contain viva notes, reproducibility details and the key source code.

2 Background

2.1 Reinforcement learning

In reinforcement learning an *agent* learns by interacting with an *environment*. At each time step t the agent observes the state s_t, chooses an action a_t, and the environment responds with a reward r_{t+1} and a new state s_{t+1} (Figure 1). The agent is not told which action is correct; it must discover which actions lead to high reward by trying them. Two features make RL different from other kinds of learning: rewards can be *delayed* (an action may only pay off many steps later), and the agent must balance *exploring* new actions against *exploiting* what it already knows.

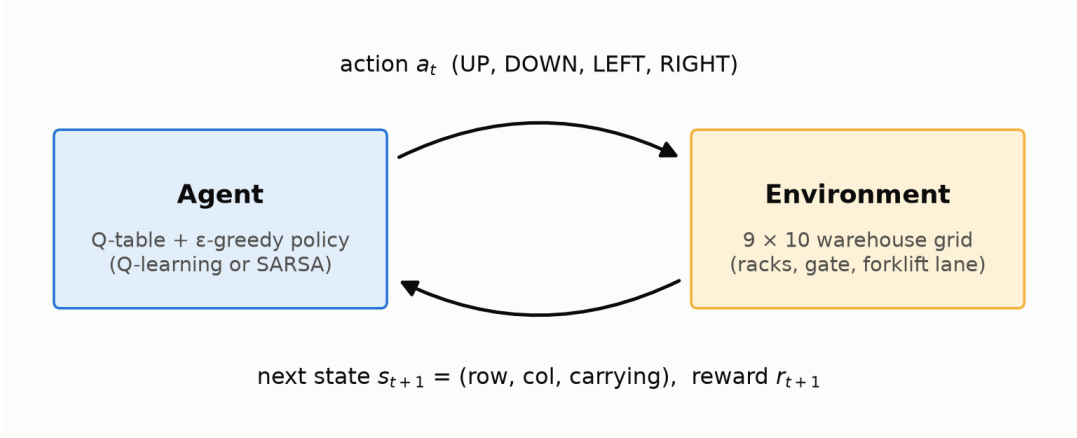


Figure 1. The agent-environment loop in this project. The agent only ever sees states and rewards; it never sees the map.

2.2 Markov decision processes

The standard mathematical model for such problems is the Markov decision process (MDP): a set of states S, a set of actions A, transition probabilities P(s′ | s, a), a reward function R and a discount factor $\gamma \in [0, 1]$. The key assumption is the *Markov property*: the next state and reward depend only on the current state and action, not on the earlier history. A policy π tells the agent which action to take in each state. The agent's goal is to maximise the expected *return*, the discounted sum of future rewards:

$$G_t = r_{t+1} + \gamma\, r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k\, r_{t+k+1} \tag{1}$$

The discount factor makes rewards that arrive sooner worth more than rewards that arrive later, and it keeps the sum finite. A γ close to 1 makes the agent far-sighted, which matters in our task because the main reward arrives about 30 moves after the start.

2.3 Action values and the Bellman optimality equation

The action-value function $Q^{\pi}(s, a)$ is the expected return when the agent takes action a in state s and follows policy π afterwards. The optimal action-value function Q* is the largest such value over all policies, and it satisfies the Bellman optimality equation:

$$Q^{*}(s,a) = \mathrm{E}\left[r + \gamma \max_{a'} Q^{*}(s', a') \;\middle|\; s, a \right] \tag{2}$$

In words: the value of an action is the immediate reward plus the discounted value of the best action in the next state. Once Q* is known, acting optimally is easy: in every state, pick the action with the largest Q-value. The learning problem is therefore to estimate Q* from experience.

2.4 Temporal-difference learning

Temporal-difference (TD) methods learn from each single step, without waiting for an episode to end and without a model of the environment. After a step, the agent compares its current estimate with a better, one-step-ahead estimate built from the reward it just received. The difference is the *TD error*:

$$\delta_t = r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \tag{3}$$

The estimate is then moved a fraction α (the learning rate) of the way towards the target. Using an estimate to update another estimate is called *bootstrapping*; it is what lets reward information spread backwards through the state space, one step per update.

2.5 Q-learning

Q-learning (Watkins, 1989) applies the TD idea directly to the Bellman optimality equation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s,a) \right] \tag{4}$$

The target uses the *maximum* over next actions, whatever action the agent actually takes next. Q-learning therefore learns the value of the optimal policy even while it behaves differently, for example while exploring at random. This is called *off-policy* learning. For a finite MDP, tabular Q-learning converges to Q* with probability 1, provided every state-action pair keeps being visited and the learning rate decreases appropriately (Watkins and Dayan, 1992).

2.6 SARSA

SARSA (Rummery and Niranjan, 1994) is the *on-policy* counterpart. Its name comes from the five values used in each update: s, a, r, s′, a′. The target uses the action a′ that the agent will really take next:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[\, r + \gamma\, Q(s',a') - Q(s,a) \,\right] \tag{5}$$

SARSA therefore learns the value of the policy it is actually following, exploration included. If that policy sometimes takes random actions, SARSA's values account for the cost of those random actions.

2.7 Exploration with ε-greedy

Both agents choose actions *ε-greedily*: with probability ε they take a uniformly random action (explore), and otherwise the action with the highest current Q-value (exploit). Ties between equal Q-values are broken at random. Early in training the Q-table is uninformative, so ε starts high; it is then reduced after every episode so that the agent increasingly uses what it has learned. We keep a small minimum ε so that the agent never stops checking alternatives while it trains.

2.8 On-policy vs. off-policy: the cliff-walking example

Sutton and Barto (2018, Example 6.6) illustrate the difference with a grid world where the shortest path runs along the edge of a cliff. With ε-greedy exploration, Q-learning learns the optimal path right along the edge, but its occasional random steps make it fall off during training. SARSA learns a longer, safer path away from the edge, because its values include the risk of its own random steps. Our warehouse contains the same structure, with the forklift lane playing the role of the cliff, and Section 5 shows that the two algorithms behave exactly as the theory predicts.

3 Problem formulation

3.1 Scenario

A delivery robot starts at its charging dock (S). It must drive to the pickup point (P) to collect a parcel, and then deliver the parcel to the packing station (D). The warehouse floor contains storage racks and a safety barrier, which the robot cannot drive through. Along the bottom of the warehouse runs a forklift lane: if the robot drives into it, that is a collision and the episode ends with a large penalty.

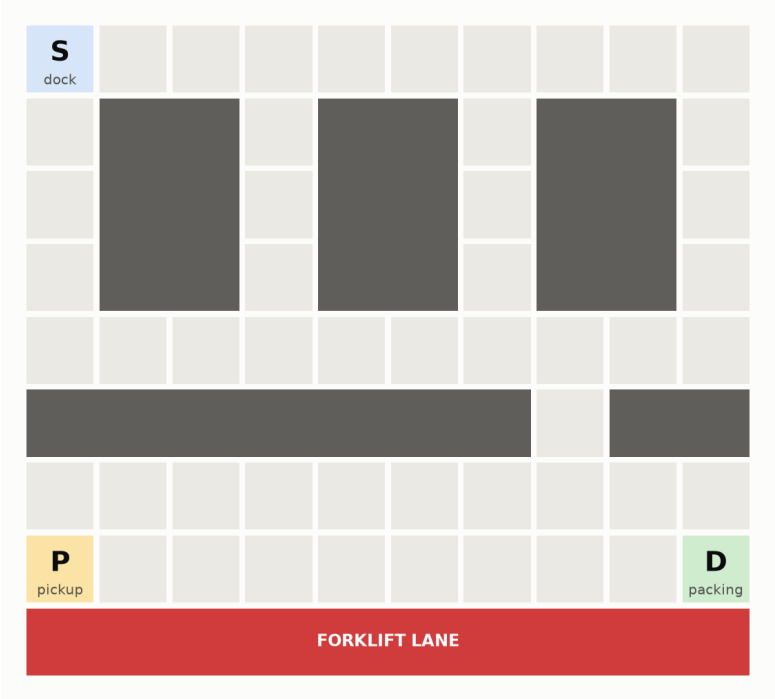


Figure 2. The warehouse used in all experiments: 9 rows × 10 columns. Dark blocks are storage racks and the safety barrier (its only gate is in column 7); the red row is the forklift lane.

3.2 Why this layout

The layout was designed so that the robot has to make genuine decisions rather than walk in a straight line:

1. **A forced detour.** The parcel is directly below the dock, but the barrier blocks the way and its only gate is on the far right. The robot must first move *away* from its target. A greedy rule such as "always move towards the goal" would get stuck pressing against the barrier.
2. **Two stages.** Before the pickup the robot must head for P; afterwards, for D. The same cell may need different actions in the two stages.
3. **Risk beside the shortest route.** P and D both sit on the row next to the forklift lane. The quickest way from P to D runs along that row (9 moves); a route one row further away is safer but takes 11 moves.

3.3 States, actions and transitions

Table 1. The warehouse task as a Markov decision process.

Element	Definition in this project
Agent	The robot's controller: a Q-learning or SARSA learner (agents.py).
Environment	The 9 × 10 warehouse grid (warehouse_env.py).
State s	(row, col, carrying), where carrying is 0 before the pickup and 1 after it. There are 53 floor cells, so 53 × 2 = 106 states.
Actions a	UP, DOWN, LEFT, RIGHT.
Transitions	Deterministic: the robot moves one cell in the chosen direction. Moving into a rack, the barrier or the outer wall leaves it where it is. Optional slippery floor: with probability p the robot slides to one of the two perpendicular directions instead.
Discount γ	0.99.
Episode end	The parcel is delivered (success), the robot enters the forklift lane (failure), or 200 moves have been used (time-out).
Objective	Maximise the expected discounted return: deliver as quickly as possible without collisions.

3.4 Reward design

Exactly one of the following rewards is given after each move:

Table 2. Reward function.

Event	Reward	Reason
Ordinary move	−1	Every move costs time and battery, so shorter routes earn more.
Bump into a rack, the barrier or the wall	−5	Hitting shelves can damage goods; it also teaches the agent to avoid pointless actions quickly.
Pick up the parcel (first arrival at P)	+20	A progress signal for the first stage. It can only be collected once per episode, so it cannot be farmed.
Deliver the parcel at D	+100	The goal. The episode ends.
Enter the forklift lane	−100	A safety-critical failure. The episode ends.

With these rewards, the best possible episode takes 30 moves: 28 ordinary moves, the pickup and the delivery.

$$G_{\text{best}} = 28 \times (-1) + 20 + 100 = 92 \tag{6}$$

3.5 The Markov property and the carrying flag

The state includes whether the robot is carrying the parcel. Without this flag the state would not be Markov: at the gate, for example, the right move depends on whether the robot has already collected the parcel, which the position alone does not reveal. With the flag, the state contains everything needed to choose the best action, and the problem is a proper MDP.

3.6 Termination and truncation

The implementation distinguishes two ways an episode can stop. *Termination* (delivery or collision) is a real end of the task, so the value of what follows is zero. *Truncation* at the 200-move limit is only a practical cut-off: the state the robot is in is not a terminal state. The update therefore still bootstraps from Q(s′, ·) after a time-out. Treating time-outs as terminal would teach the agent a wrong lesson about the states it happens to be in when time runs out.

3.7 Slippery-floor variant

To test robustness under uncertainty, the environment has an optional slip probability p. With probability p the robot's wheels slip and it moves to one of the two directions perpendicular to the intended one. With p = 0.1, a robot driving along the row next to the forklift lane has a 5% chance per move of sliding into the lane, so the risk of that route becomes real rather than being caused only by exploration.

4 System design and implementation

4.1 Architecture

The system is written in Python with NumPy and Matplotlib only, in five small modules (Figure 3). **warehouse_env.py** defines the environment. **agents.py** defines the learners. **train.py** runs the interaction loop, the experiments and the tests, and saves results. **plots.py** draws every figure, and **demo.py** animates trained agents for the live demonstration.

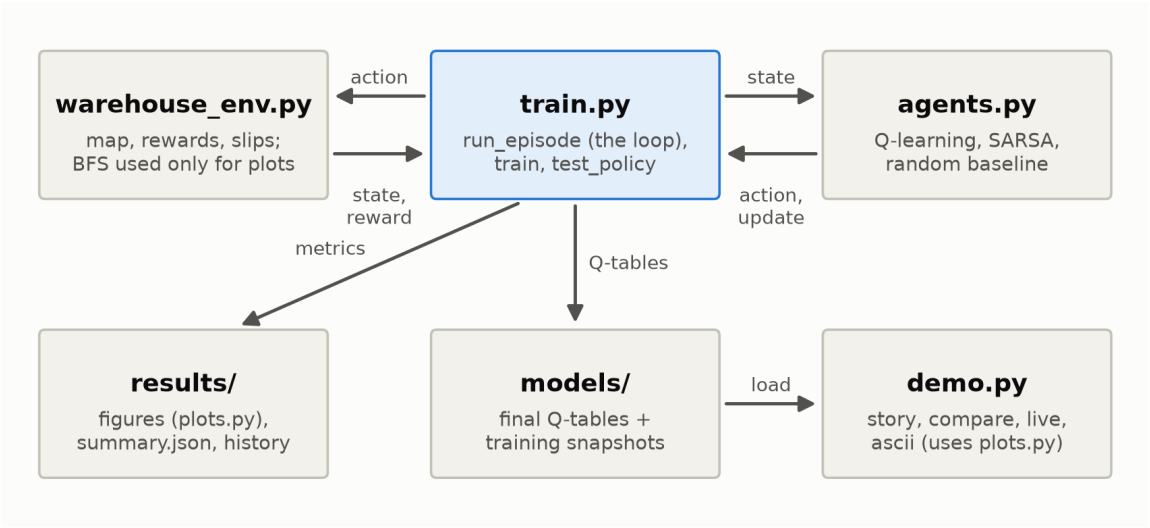


Figure 3. Module structure and data flow. train.py is the hub: it connects the environment and the agents, and produces the results and saved Q-tables that the demo uses.

4.2 The environment

The warehouse is described as a short block of text (one character per cell: S, P, D, #, ~ and .), so the layout can be changed without touching the code. The step function applies an action, handles slips and blocked moves, assigns the reward and reports whether the episode terminated or was truncated, in the same five-value format used by the Gymnasium library. The module also contains a breadth-first search that computes the shortest possible route length; it is used only to draw the “best possible” reference lines on the plots, and the agents never call it.

4.3 The agents

Both learning agents share a base class that stores the Q-table as a NumPy array of shape 9 × 10 × 2 × 4 (rows × columns × carrying × actions), initialised to zero. It implements ε-greedy action selection with random tie-breaking and the per-episode ε decay. The two algorithms differ only in the target of their update method (Listing 1), so any difference in behaviour comes from the update rule alone. A random agent with the same interface serves as the baseline.

Listing 1. The complete difference between the two learners (agents.py). Q-learning bootstraps from the best next action; SARSA from the next action it will actually take.

```
def update(self, state, action, reward, next_state, next_action, terminated):
    future = 0.0 if terminated else self.gamma * self.q[next_state].max()
    td_error = reward + future - self.q[state][action]
    self.q[state][action] += self.alpha * td_error
    return td_error

def update(self, state, action, reward, next_state, next_action, terminated):
    future = 0.0 if terminated else self.gamma * self.q[next_state][next_action]
    td_error = reward + future - self.q[state][action]
    self.q[state][action] += self.alpha * td_error
    return td_error
```

4.4 The interaction loop

A single function, run_episode, implements the agent-environment loop of Figure 1 and is used for training, testing and recording (Listing 2). It chooses the next action before calling the update, because SARSA needs it, and passes only the *terminated* flag to the update, so time-outs still bootstrap. With record=True it also returns every state, event and reward, which the figures and demos replay.

Listing 2. The agent-environment loop (train.py).

```
def run_episode(env, agent, learn=True, greedy=False, record=False):
    """Play one episode: the agent-environment interaction loop.

    learn=True:   update the Q-table after every step (training)
    greedy=True:  always take the best known action, no exploration (testing)
    record=True:  also return every state, event and reward, for figures and the demo
    """
    choose = agent.greedy_action if greedy else agent.act
    state = env.reset()
    action = choose(state)
    total_reward = 0
    states, events, rewards, slips = [state], [], [], []
    while True:
        next_state, reward, terminated, truncated, info = env.step(action)
        next_action = choose(next_state)  # SARSA's update needs the next action
        if learn:
            agent.update(state, action, reward, next_state, next_action, terminated)
        total_reward += reward
        if record:
            states.append(next_state)
            events.append(info["event"])
            rewards.append(reward)
            slips.append(info["slipped"])
        state, action = next_state, next_action
        if terminated or truncated:
            break
    result = {"reward": total_reward, "steps": env.steps,
              "outcome": info["event"] if terminated else "timeout"}
    if record:
        result.update(states=states, events=events, rewards=rewards, slips=slips)
    return result
```

4.5 Experimental protocol

- **Ten independent runs per algorithm.** Each run uses its own random seed, so results are reported as the mean ± standard deviation over ten runs rather than one possibly lucky run.
- **Training.** 1,000 episodes per run on the normal floor and 2,000 on the slippery floor.
- **Tests during training.** Every 10 episodes the current greedy policy is tested with exploration and learning switched off. The test borrows a separate random-number generator, so testing never changes the course of training.
- **Final test.** Each trained agent plays 100 test episodes with exploration and learning switched off.
- **Baseline.** A random agent is tested in exactly the same way.
- **Metrics.** Cumulative reward (return) per episode, success rate (parcel delivered), number of steps, collision and time-out rates, and the episode from which the learned route stays reliable.
- **Reproducibility.** Every random number comes from a seeded generator, so re-running train.py reproduces every number in this report exactly.

4.6 Hyperparameters

Table 3. Hyperparameters and the reasons for each value.

Parameter	Value	Justification
Learning rate α	0.1	A standard step size for tabular TD learning; the route is learned in about 300 episodes. A decaying α was also tested and needed up to 3× more episodes.
Discount factor γ	0.99	An optimal run takes 30 moves. With γ = 0.95 the +100 delivery reward would be worth only $0.95^{29} \approx 0.23$ of its value at the start; with 0.99 it keeps ≈ 0.75, so the distant goal still matters.
Exploration ε	1.0 → 0.05	Start fully random; multiply by 0.995 after every episode, which reaches the minimum of 0.05 around episode 600.
Training episodes	1,000 (2,000 slippery)	The route is reliable after about 300 episodes; the rest shows the stable plateau. Random slips need more experience.
Step limit	200 moves	Ends endless wandering early in training.
Independent runs	10 per algorithm	Gives means and standard deviations instead of single runs.

4.7 Visualisation and demonstration

plots.py draws all result figures with one consistent colour scheme (Q-learning in blue, SARSA in orange, the random agent in grey) and provides the warehouse drawing that the demo reuses, so figures and animations look identical. demo.py has four modes: a *story* that replays the same robot before, part-way through and after training; a side-by-side *comparison* of Q-learning and SARSA; *live* training of a new agent with its learning curve growing on screen; and a terminal (*ASCII*) mode that needs no window. Any mode can also be saved as an animated GIF.

5 Results

All numbers below are produced by train.py and stored in results/summary.json. Unless stated otherwise, results are means ± standard deviations over 10 training runs, and “test” means 100 episodes with exploration and learning switched off. The full experiment (20 training runs plus all tests) took 3.3 seconds on a laptop.

5.1 Final test performance

Table 4. Final test with exploration and learning switched off.

Agent	Delivery success	Moves per episode	Return per episode
Random agent (no learning)	0.0%	188.3 ± 2.7	-555.5 ± 5.9
Q-learning	100.0%	30.0 ± 0.0	92.0 ± 0.0
SARSA	100.0%	32.0 ± 0.0	90.0 ± 0.0
Best possible (BFS reference)	100%	30	92

Both trained agents deliver the parcel in every test episode of every run. Q-learning uses exactly 30 moves, the shortest possible route, and earns the best possible return of 92. SARSA uses 32 moves. The random agent never delivered in 1,000 test episodes: it either ran out of moves or drove into the forklift lane (17.8% of episodes).

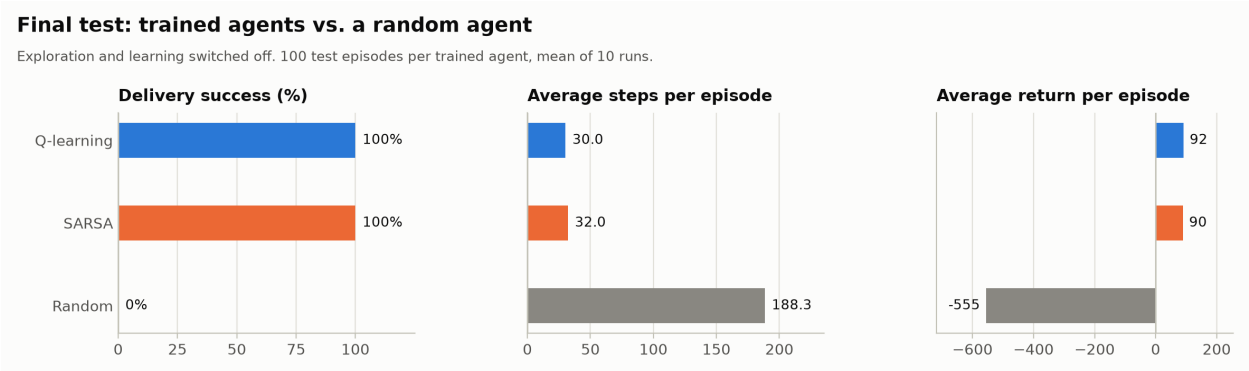


Figure 4. Final test results: trained agents compared with the random agent.

KEY RESULT

Starting from an all-zero Q-table and without ever seeing the map, the Q-learning robot learned the shortest possible delivery route (30 moves) in all 10 independent runs.

5.2 Learning curves

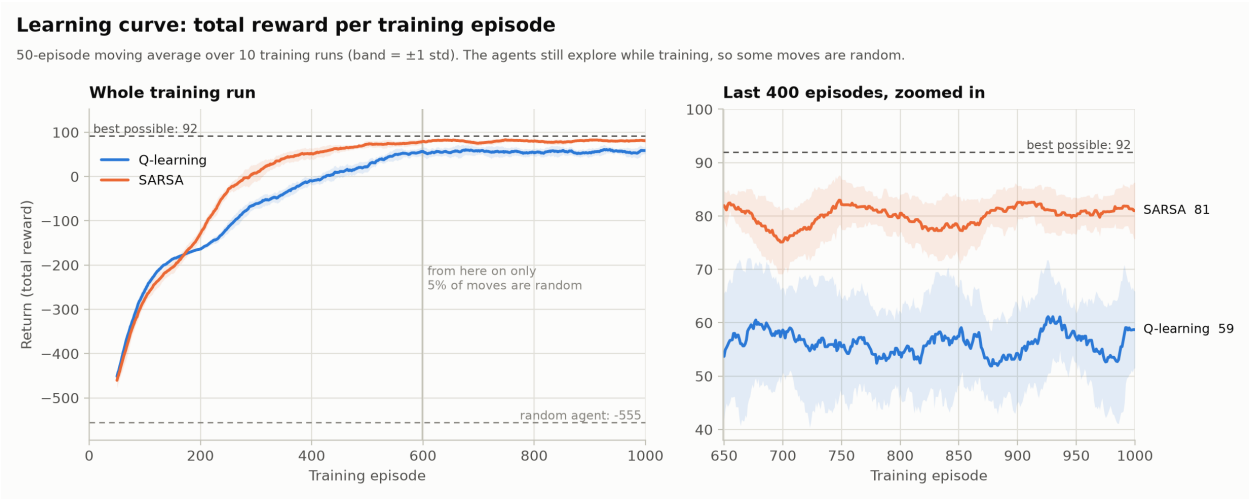


Figure 5. Return per training episode (50-episode moving average, mean of 10 runs, band = ±1 standard deviation). Right: the last 400 episodes, zoomed in.

Both agents begin at the level of the random agent and climb towards the best possible return as ε decays (Figure 5). The zoomed panel shows a difference that is easy to miss: during training, SARSA earns more reward than Q-learning. Over the last 100 training episodes SARSA averages 80.5 and Q-learning 57.9. Q-learning has not learned less; its route simply runs beside the forklift lane, where the 5% of moves that are still random sometimes cause a collision.

5.3 Steps per episode and success rate

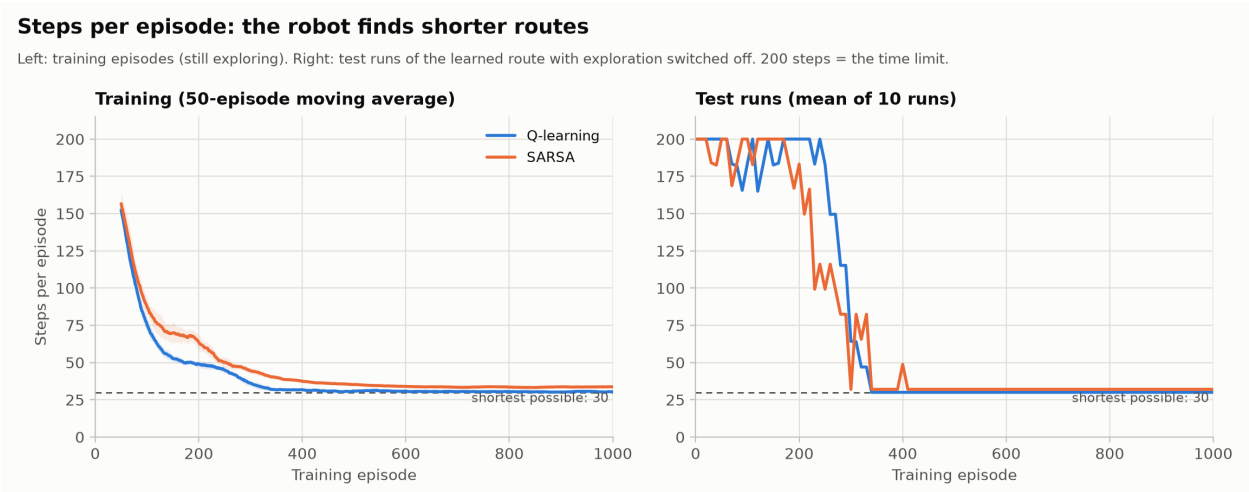


Figure 6. Steps per episode. Left: training episodes (still exploring). Right: test runs of the learned route with exploration switched off.

With exploration switched off (Figure 6, right), the learned route drops from the 200-move time limit to exactly 30 moves for Q-learning and 32 for SARSA. The point from which every later test delivers was reached after 270–340 episodes for Q-learning (mean 303) and 260–410 for SARSA (mean 311).

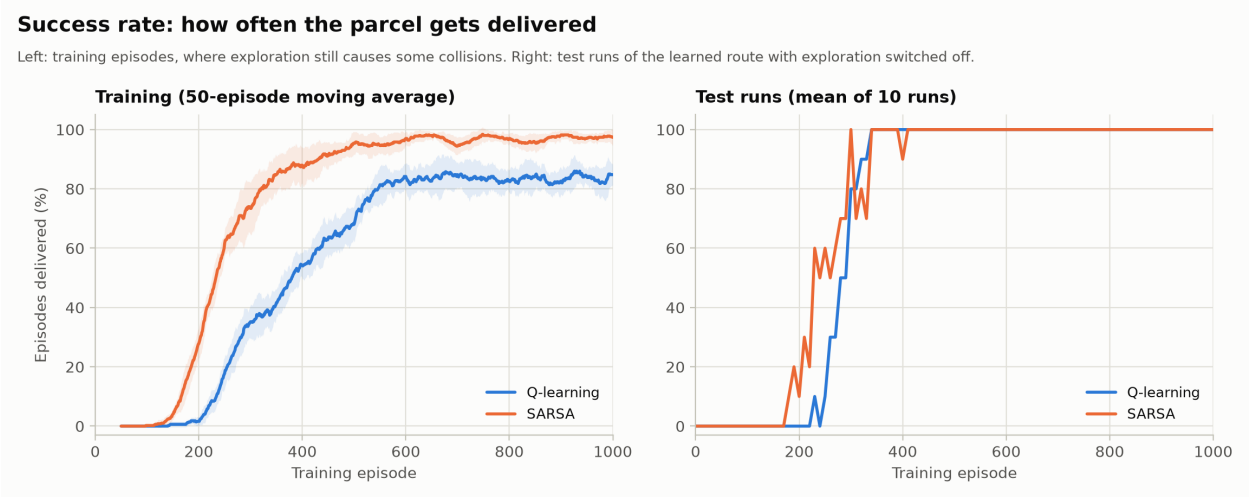


Figure 7. Success rate. Left: share of training episodes delivered. Right: share of runs whose learned route delivers in the test.

During training (Figure 7, left), SARSA's success rate settles at about 97.2% and Q-learning's at about 84.4%. In the tests (right), both reach 100% and stay there.

5.4 How training episodes ended

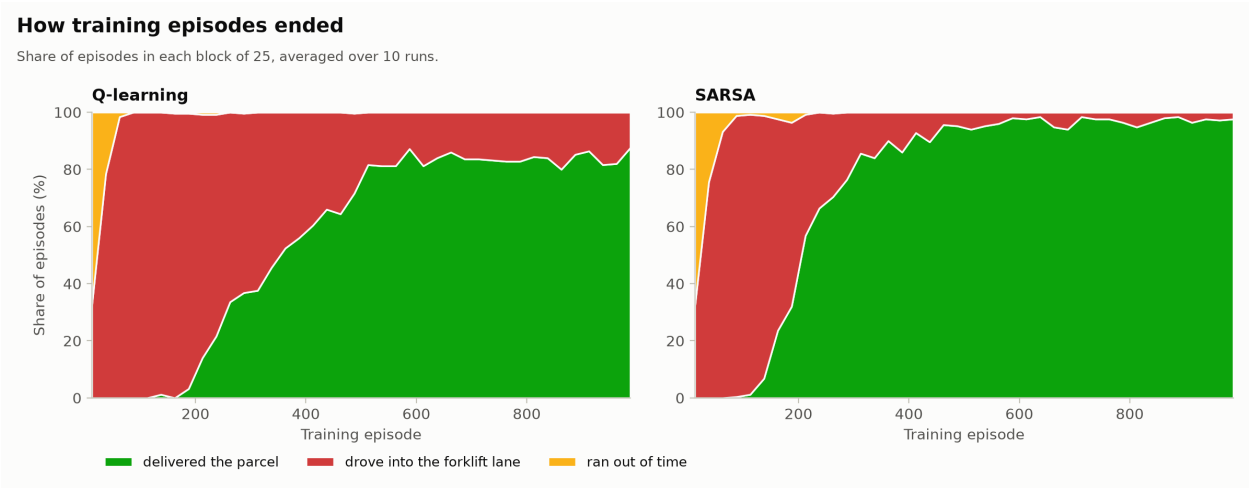


Figure 8. Share of training episodes that ended in a delivery, a collision or a time-out, per block of 25 episodes, averaged over 10 runs.

Early in training, almost every episode ends in a time-out. Soon after, most episodes end in the forklift lane: the agent has learned to get through the gate but still moves mostly at random near the lane. As ε decays, deliveries take over. With 5% exploration left, Q-learning still collides in 15.6% of training episodes and SARSA in 2.8%; over a whole run, Q-learning collides 418 times on average and SARSA 226 times.

5.5 Learning progress of a single robot



Figure 9. The learned route of the same Q-learning robot (run 0) at twelve points during training, each replayed with exploration switched off.

Figure 9 shows the learning process directly. Before training the robot wanders at random. From episode 25 to 200 its greedy choice is to move back and forth on the same spot. This is expected: reward information spreads backwards one step per update, and until the +100 at the packing station has propagated back to the dock, all moves near the start look equally bad. By episode 250 the robot fetches the parcel but then loops; from episode 300 onward it drives the shortest route in every test.

5.6 Learned routes

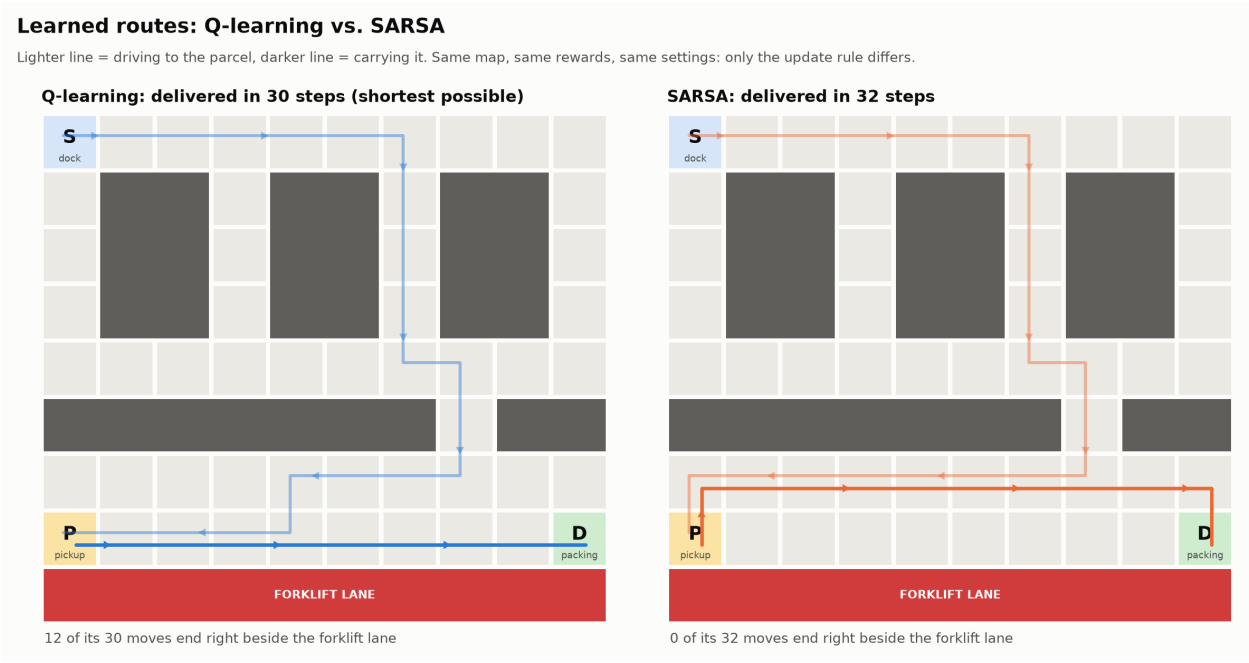


Figure 10. Final routes of the two agents (run 0). Lighter line: driving to the parcel; darker line: carrying it.

Q-learning's route carries the parcel along the row right beside the forklift lane: 12 of its 30 moves end next to the lane. SARSA's route keeps one row away: none of its 32 moves end next to the lane (Figure 10). This is the cliff-walking effect of Section 2.8, reproduced in the warehouse.

5.7 Inside the Q-tables

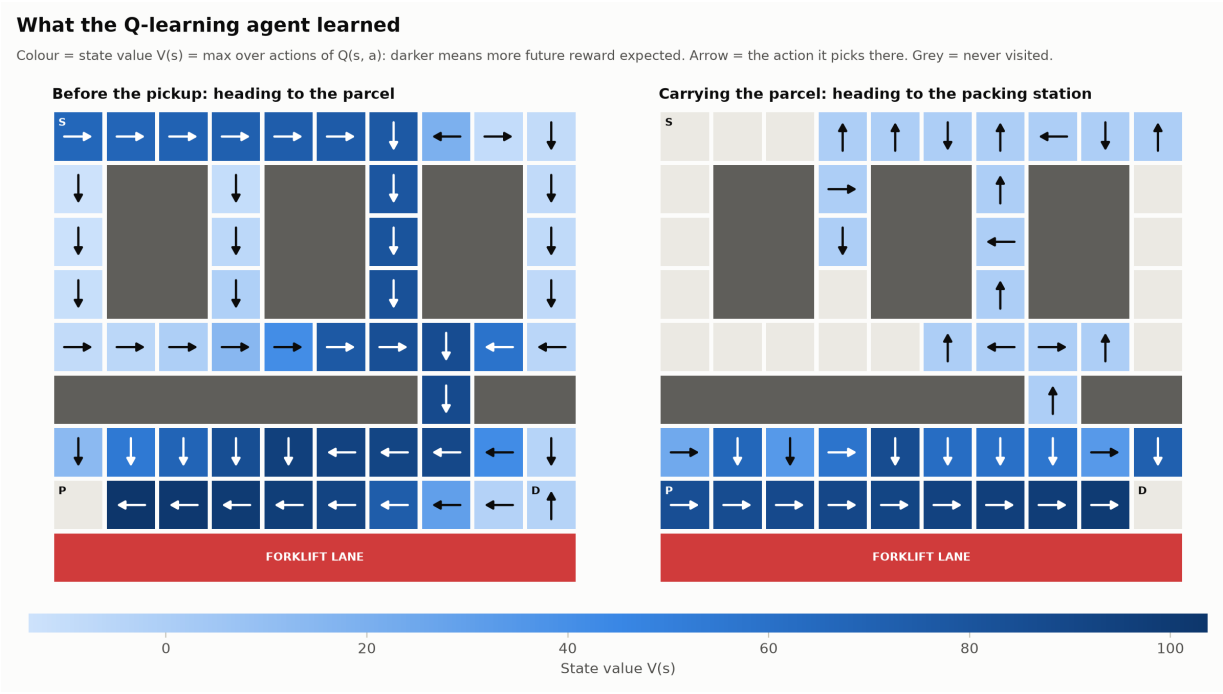


Figure 11. What the Q-learning agent learned. Colour: state value V(s) = $\max_a$ Q(s, a); arrows: the chosen action; grey: never visited.

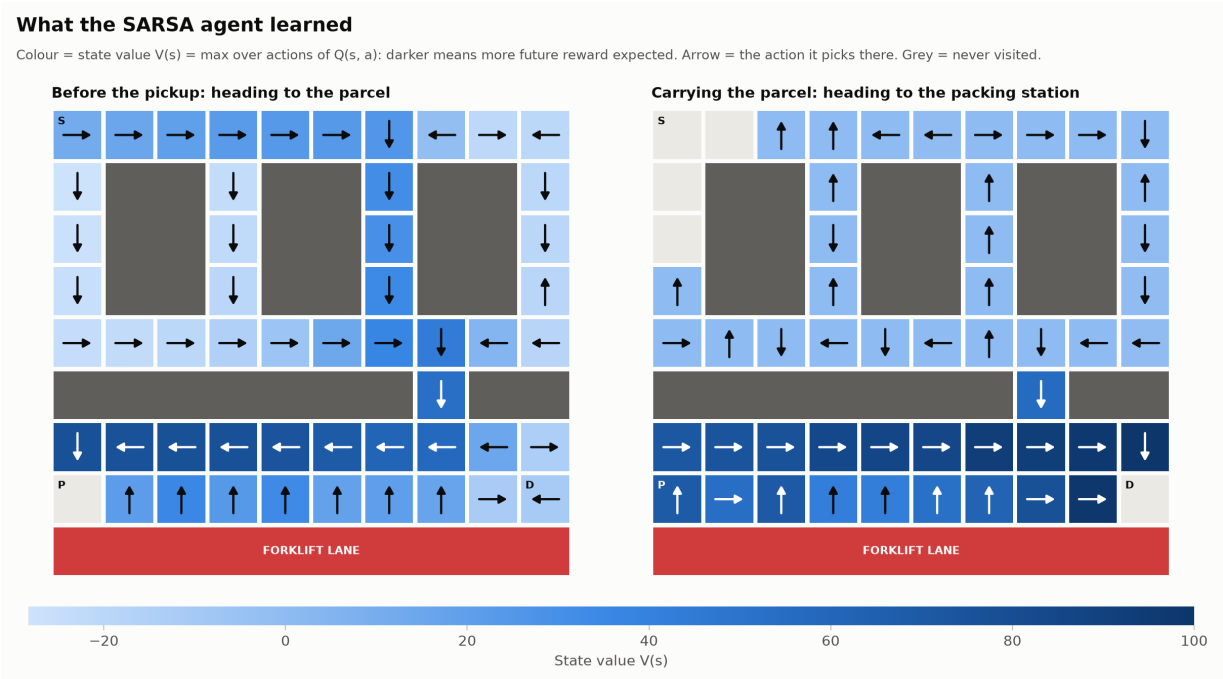


Figure 12. What the SARSA agent learned, in the same format.

Because the agents are tabular, their knowledge can be inspected directly (Figures 11 and 12). Values rise towards the goal of each stage, and the arrows form a route through the gate. SARSA's map shows its caution: while carrying the parcel on the row next to the lane, most of its arrows point *up*, away from the lane (only the cells closest to the packing station go straight there), and that row has lower values than the row above. SARSA has priced in the risk of its own random moves; Q-learning, which assumes the best move will always be taken, has not.

5.8 Slippery floor

Table 5. Slippery floor (10% of moves slide sideways): final test over 10 runs × 2,000 episodes.

Agent	Delivery success	Moves per episode	Return per episode
Random agent	0.0%	187.7 ± 4.1	-553.9 ± 9.6
Q-learning	99.3 ± 0.8%	36.8 ± 0.3	75.5 ± 1.7
SARSA	99.1 ± 1.2%	36.9 ± 0.3	75.2 ± 2.0

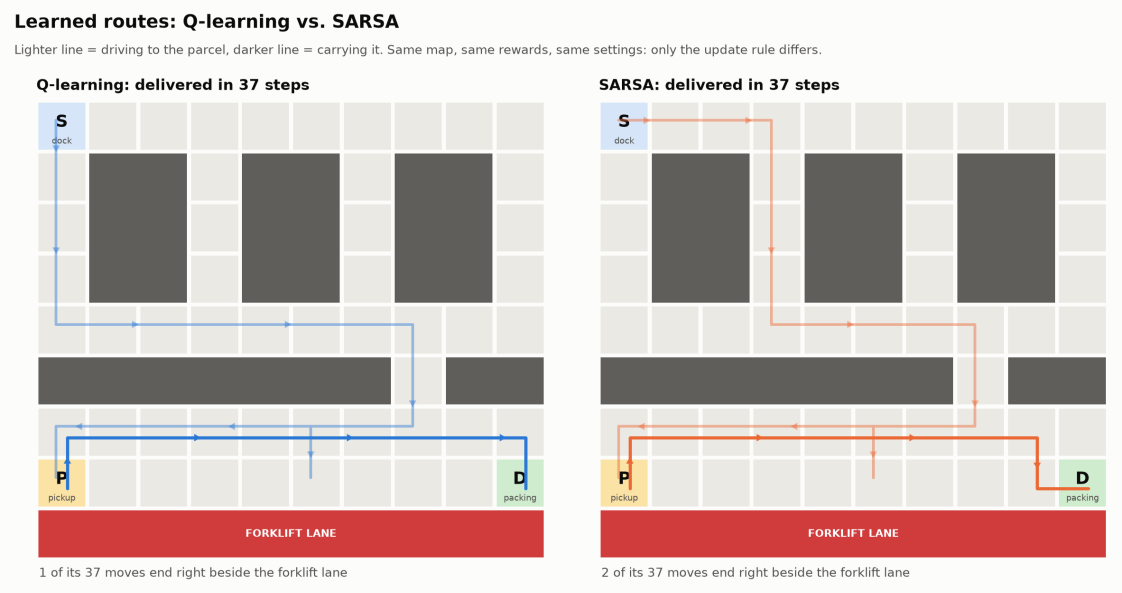


Figure 13. Learned routes on the slippery floor (one sampled episode each, including slips).

On the slippery floor, a robot driving beside the lane has a real 5% chance per move of sliding into it. Both algorithms now learn to carry the parcel along the row one away from the lane, Q-learning included (Figure 13), and both deliver in about 99% of tests. This is the strongest argument for RL over a shortest-path planner: the agent optimises expected reward under uncertainty, and the best route is no longer the shortest one.

6 Discussion

6.1 Q-learning vs. SARSA: speed against safety

Both algorithms solve the task, but they answer a slightly different question. Q-learning learns the best route *assuming every move is executed perfectly*; SARSA learns the best route *given that the agent still makes random moves*. Which is preferable depends on deployment. A robot that follows its learned route exactly should use Q-learning's 30-move route. A robot that keeps exploring, or whose movements are imprecise, is better served by SARSA's caution, which costs only two moves. In our data this caution cut training collisions from 15.6% to 2.8% of episodes.

6.2 Why reinforcement learning rather than path planning

A shortest-path algorithm such as A* or breadth-first search could find the 30-move route instantly, so it is fair to ask why RL is needed. There are three reasons. First, A* requires the map and the costs in advance; our agent receives neither and learns only from interaction. Second, RL maximises expected reward, which includes risk: on the slippery floor the shortest route is no longer the best one, and both agents learned that. Third, the same code learns any layout: when we changed the warehouse (moving the gate, the parcel and the packing station), Q-learning found the new 29-move optimum with no code changes.

6.3 Evidence that the behaviour is learned

- The Q-table starts at all zeros; the agent has no map, no route and no knowledge of the rewards.
- The only information it receives is what the environment returns after each move: the next state and the reward.
- The breadth-first search in the code is used only for the reference lines on the plots.
- Snapshots of a single robot (Figure 9) show the behaviour improving step by step, and the live demo trains a new agent from scratch in front of the audience.
- All ten independent runs of each algorithm converge, and a different layout is learned without code changes.

6.4 Limitations

- **Tabular representation.** A table needs one entry per state-action pair. This is ideal for 106 states but does not scale to large warehouses or continuous sensor input, where function approximation (e.g. a DQN) is needed.
- **Constant learning rate under randomness.** With a constant α, Q-values keep fluctuating on the slippery floor, and occasionally a run ends with a greedy policy that loops. We observed this in 1 of 10 Q-learning runs at 1,000 episodes and 2 of 10 SARSA runs at 3,000 episodes, and in none at the default of 2,000. Simply decaying α did not help: its worst slippery-floor runs reached only 82–92% success, against at least 94.5% with a constant α, and it slowed learning on the normal floor.

- **A static world.** The forklift lane is fixed; real forklifts move. There is one robot, one parcel per trip and no battery model.
- **Simulation only.** The grid abstracts away real motion, sensing and localisation.

7 Demonstration and usage guide

7.1 Installation

The project needs Python 3.9 or newer, NumPy and Matplotlib. From the project folder:

Listing 3. Installing and running the experiments.

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt

python train.py                    # train + test both agents, write results/ and models/
python train.py --slip 0.1         # slippery-floor experiment, results/slip_0.10/
```

7.2 Demo modes

Table 6. Demonstration commands. Add --slip 0.1 to use the slippery-floor agents.

Command	What it shows
python demo.py	The same robot before training, part-way through and fully trained
python demo.py --compare	Q-learning and SARSA side by side on the same map
python demo.py --live	A brand-new robot training on screen while its learning curve grows (about 20 s)
python demo.py --ascii	The story in the terminal, with no window needed
python demo.py --save demo.gif	Saves any mode as an animated GIF instead of opening a window

If no window opens, Matplotlib has no GUI toolkit; on Ubuntu, install one with sudo apt install python3-tk. In a demo window, press f for full screen and q to close it.

7.3 What the demos look like

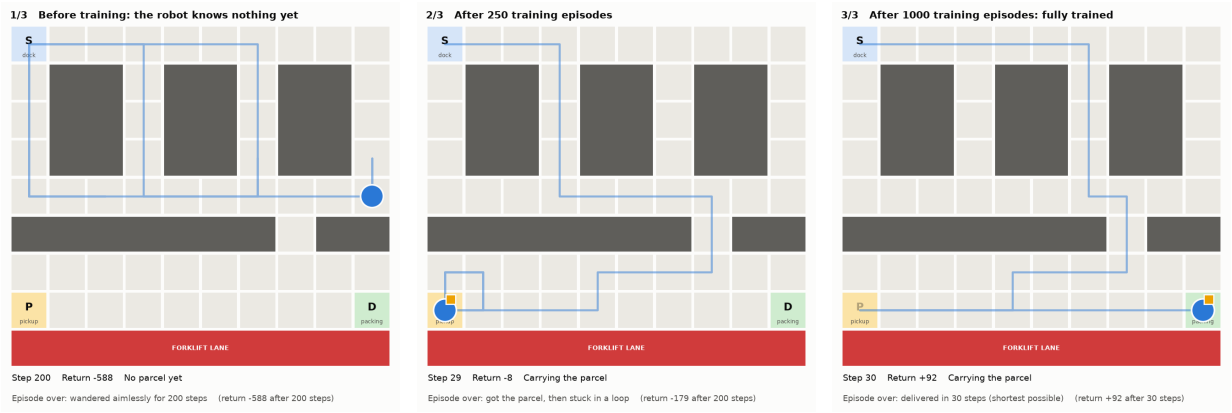


Figure 14. Story mode (python demo.py): the same robot before training (wanders until time runs out), after 250 episodes (fetches the parcel, then loops) and after 1,000 episodes (shortest route).

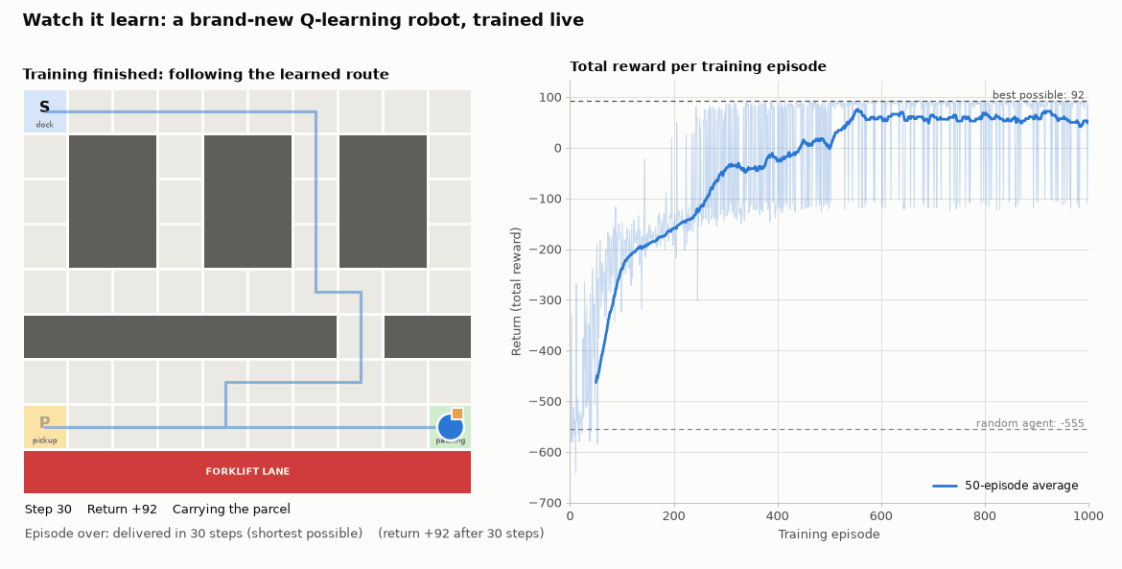


Figure 15. Live mode (python demo.py --live) at the end of training: the learned route on the left, the reward curve of the whole training run on the right.

8 Implemented features and future enhancements

As required by the brief, this section separates what exists and works today from what is proposed.

Table 7. Implemented features compared with proposed future enhancements.

Implemented and working	Proposed future enhancements (not implemented)
Warehouse grid world: racks, a barrier with one gate, a forklift-lane hazard, a two-stage pickup-and-delivery task, a step limit and an optional slippery floor	Deep Q-Network (DQN) with experience replay and a target network, for warehouses too large for a table or for camera input
Tabular Q-learning and SARSA with ε-greedy exploration and ε decay; random-agent baseline	Prioritised experience replay, to learn faster from rare but important events such as collisions
Ten seeded runs per algorithm, a greedy test every 10 episodes and a final 100-episode test	Double Q-learning or Expected SARSA, and per-state learning-rate schedules, to remove the occasional looping policy on the slippery floor
Slippery-floor robustness experiment	Moving forklifts (dynamic obstacles), several parcels per trip and battery level in the state
Nine evaluation figures, summary.json, the full training history and saved Q-tables with training snapshots	Several robots sharing the floor (multi-agent reinforcement learning)
Four demonstration modes and GIF export	A ROS/Gazebo simulation, followed by tests on a physical robot

9 Conclusion

We built a reinforcement-learning agent that learns to operate a warehouse delivery robot from rewards alone. Formulated as a 106-state MDP, the task requires a detour, two stages and a trade-off between speed and safety. Tabular Q-learning learned the shortest possible 30-move route in all ten independent runs, after about 300 episodes, while a random agent never delivered. SARSA learned a route two moves longer that keeps away from the forklift lane, reproducing the textbook contrast between off-policy and on-policy learning, and on a slippery floor both agents learned to take the safer route.

The project shows the core ideas of reinforcement learning in a form that can be watched, measured and questioned: an agent with no prior knowledge, a reward signal that spreads backwards through the Bellman updates, and a learned policy that adapts to risk. The proposed extensions, from deep Q-networks to moving forklifts and multiple robots, are the natural next steps towards a real warehouse.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[2] C. J. C. H. Watkins, *Learning from Delayed Rewards*, PhD thesis, University of Cambridge, 1989.

[3] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, pp. 279–292, 1992.

[4] G. A. Rummery and M. Niranjan, "On-line Q-learning using connectionist systems," Technical Report CUED/F-INFENG/TR 166, Cambridge University Engineering Department, 1994.

[5] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[6] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

Acknowledgement: this project was developed with help from Claude (Anthropic), an AI assistant.

Appendix A Viva preparation: likely questions

What exactly does the agent learn?

A table with one Q-value per state-action pair: 106 reachable states × 4 actions. Q(s, a) estimates the total discounted reward of taking action a in state s and acting well afterwards. The policy picks the action with the highest Q-value.

Where does the Bellman equation come in?

The target $r + \gamma \cdot \max Q(s', \cdot)$ is a one-step look-ahead taken from the Bellman optimality equation. The TD error is that target minus the current estimate, and each update moves Q(s, a) 10% (α) of the way towards the target.

What is the difference between Q-learning and SARSA?

Only the target. Q-learning uses the best next action (off-policy); SARSA uses the next action it will really take, including random ones (on-policy). That is why SARSA keeps away from the lane.

Why does ε decay, and why not all the way to zero?

At the start the agent knows nothing, so exploring is valuable; later it should mostly use what it has learned. Keeping 5% lets it keep correcting its estimates while training. Tests switch exploration off completely.

Why is Q-learning's success about 84% during training but 100% in the test?

During training 5% of moves are random and its route runs beside the lane, so a random DOWN move there is a collision. With exploration off, that mistake never happens.

Why did the robot go back and forth early in training?

Reward information spreads backwards one step per update. Until the +100 at D has propagated back to the start, every nearby move looks equally bad (each has only ever cost −1), so the greedy choice bounces between neighbouring cells.

Why does the state include "carrying"?

At the same cell the right move differs before and after the pickup. Without the flag the state would not be Markov.

Why the +20 pickup reward?

Without it the only positive reward would be about 30 moves away, and random exploration would take much longer to find it. It can be collected once per episode, so it cannot be farmed.

What happens at a time-out?

The episode is truncated, not terminated, so the last update still includes $\gamma \cdot \max Q(s', \cdot)$: running out of time is not a property of the state.

Is convergence guaranteed?

In theory, yes: tabular Q-learning converges to Q* with probability 1 if every state-action pair keeps being visited and the step size shrinks appropriately (the Robbins–Monro conditions). With constant $\alpha = 0.1$, all ten deterministic runs reached the exact optimum.

Why not just use A* or BFS?

A* needs the map and the costs up front; our agent has neither. RL also accounts for risk, which matters on the slippery floor, and the same code learns any layout.

Why ten runs?

Results depend on random exploration. Averaging over ten independent runs and reporting the standard deviation shows the result is reliable rather than one lucky run.

What would change for a much larger warehouse?

The table grows with the number of states. With tens of thousands of states, or continuous sensor input, a function approximator such as a deep Q-network would replace the table.

Appendix B Reproducibility details

Table 8. Everything needed to reproduce the results.

Item	Value
Software	Python 3.9+ with NumPy ≥ 1.24 and Matplotlib ≥ 3.7 (Pillow, installed with Matplotlib, for GIFs)
Configuration	episodes 1000 (slippery floor 2000), runs 10, α 0.1, γ 0.99, ε 1.0 → 0.05 (×0.995 per episode), step limit 200, test every 10 episodes, 100 final test episodes
Random seeds	Run k uses environment seed k, agent seed k + 10,000, in-training test seeds k + 50,000 and k + 90,000, and final-test seeds 70,000 + k and 80,000 + k
Run time	3.3 s for the normal floor and 12.2 s for the slippery floor (all 20 runs and tests, on a laptop)
Outputs	results/: figures, summary.json, training_history.npz, demo GIFs; models/: final Q-tables and training snapshots; results/slip_0.10/ and models/slip_0.10/ for the slippery floor
Source code	github.com/KS-KARTHIK-05/Warehouse-Delivery-Robot-using-Q-Learning-and-SARSA (MIT License)

Appendix C Key source code

The listings below are copied automatically from the project files when this report is built, so they always match the submitted code. The complete code is in the repository.

Listing 4. The environment's step function (warehouse_env.py): slips, blocked moves, rewards, termination and truncation.

```
def step(self, action):
    """Try to move the robot one cell.

    Returns (next_state, reward, terminated, truncated, info):
      terminated: the task really ended (parcel delivered, or collision)
      truncated:  we ran out of time (max_steps); the state itself is not terminal
    """
    self.steps += 1
    slipped = self.slip_prob > 0 and self.rng.random() < self.slip_prob
    if slipped:
        action = SIDEWAYS[action][self.rng.integers(2)]
    dr, dc = MOVES[action]
    target = (self.pos[0] + dr, self.pos[1] + dc)

    terminated = False
    if not self._inside(target) or target in self.walls:
        reward, event = REWARD_BUMP, "bump"  # blocked, so the robot stays where it is
    else:
        self.pos = target
        if target in self.hazards:
            reward, event, terminated = REWARD_COLLISION, "collision", True
        elif target == self.pickup and not self.carrying:
            self.carrying = 1
            reward, event = REWARD_PICKUP, "pickup"
        elif target == self.dropoff and self.carrying:
            reward, event, terminated = REWARD_DELIVER, "delivered", True
        else:
            reward, event = REWARD_STEP, "move"

    truncated = not terminated and self.steps >= self.max_steps
    return self.state, reward, terminated, truncated, {"event": event, "slipped": slipped}
```

Listing 5. ε-greedy action selection with random tie-breaking, and ε decay (agents.py).

```
def greedy_action(self, state):
    """Best action according to the Q-table. Ties are broken randomly."""
    q = self.q[state]
    best = np.flatnonzero(q == q.max())
    return int(best[0] if len(best) == 1 else self.rng.choice(best))

def act(self, state):
    """Epsilon-greedy: explore with probability epsilon, otherwise exploit."""
    if self.rng.random() < self.epsilon:
        return int(self.rng.integers(self.n_actions))
    return self.greedy_action(state)

def end_episode(self):
    self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)
```

Listing 6. Testing with a borrowed random-number generator, so tests never change the training run (train.py).

```
def test_policy(agent, env, episodes, rng, record=False):
    """Play the greedy policy with learning and exploration switched off.

    The agent borrows `rng` for tie-breaking, so testing never alters the training run.
    """
    training_rng, agent.rng = agent.rng, rng
    results = [run_episode(env, agent, learn=False, greedy=True, record=record)
               for _ in range(episodes)]
    agent.rng = training_rng
    return results
```